Linked Lists & Trees Exploration

1. Project Overview

Linked Lists & Trees Exploration is a Python-based console application developed to demonstrate the implementation of advanced linear and hierarchical data structures. The project includes the implementation of **Singly Linked List, Doubly Linked List, Circular Linked List, and Binary Search Tree (BST)** along with their fundamental operations.

The project also includes an algorithm to determine whether a Binary Tree is balanced. This project provides practical exposure to **Data Structures and Algorithms (DSA)** concepts that are widely used in software development and technical interviews.

2. Problem Statement

Develop a Python application to implement advanced data structures such as **Singly Linked List, Doubly Linked List, Circular Linked List, and Binary Search Tree (BST)**.

The application should support operations such as:

- Insertion
- Deletion
- Searching
- Tree traversals
- Checking whether a Binary Tree is balanced

The project aims to provide practical understanding of **linear and hierarchical data structures** and strengthen problem-solving skills through hands-on implementation.

3. Features

🔗 Linked List Operations

- Implement Singly Linked List
- Implement Doubly Linked List

- Implement Circular Linked List
- Insert nodes at the beginning
- Insert nodes at the end
- Insert nodes at a specified position
- Delete nodes from the beginning
- Delete nodes from the end
- Delete nodes from a specified position
- Search for an element

🌳 **Binary Search Tree Operations**

- Implement Binary Search Tree
- Insert nodes into the BST
- Delete nodes from the BST
- Search for nodes in the BST
- Perform Inorder Traversal
- Perform Preorder Traversal
- Perform Postorder Traversal
- Perform Level-order Traversal

⚖️ **Balanced Binary Tree**

- Check whether a Binary Tree is balanced
- Calculate subtree heights
- Compare the heights of left and right subtrees
- Display whether the tree is balanced or unbalanced

4. Technologies Used

- Python 3
- Visual Studio Code (VS Code)

- Command Prompt / Terminal

5. Python Concepts Used

- Functions
- Classes and Objects
- Node-based data structures
- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Binary Search Tree (BST)
- Queue
- Recursion
- Searching
- Tree Traversals
- Conditional Statements
- Loops

6. Program Workflow

Start

↓

Choose Data Structure

(Linked List / Binary Search Tree)

↓

Perform Required Operation

(Insert / Delete / Search / Traversal)

↓

Display Output

↓

Check Binary Tree Balance

↓

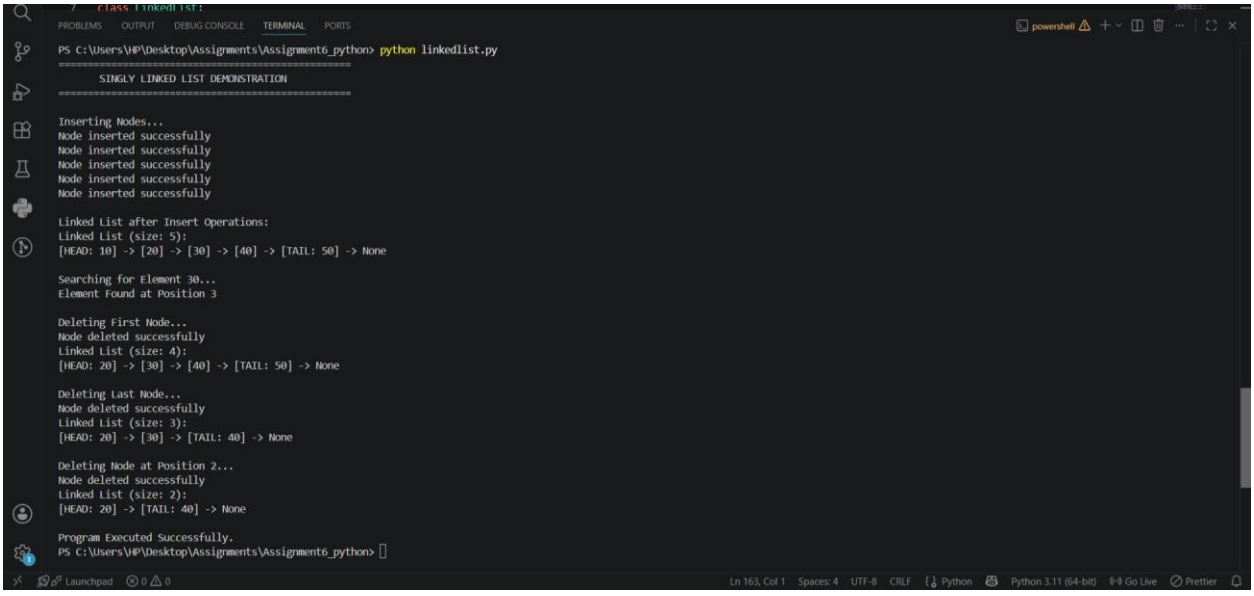
Display Result

↓

End

7. Output Screenshots and Explanation

Screenshot 1: Singly Linked List Operations

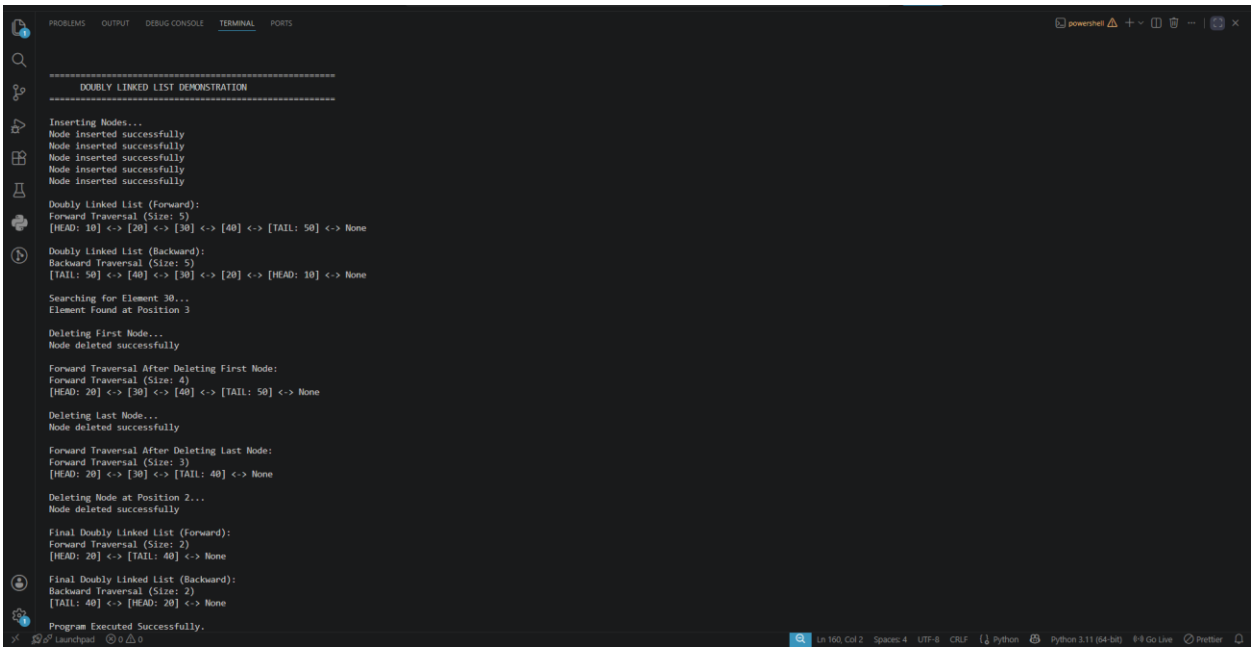


Explanation

This screen demonstrates the implementation of the Singly Linked List. The program performs insertion of nodes at the beginning, end, and a specified position. It also demonstrates deletion operations and searching for a particular element.

The output confirms that the linked list operations are performed correctly while maintaining the proper sequence of nodes.

Screenshot 2: Doubly Linked List Operations

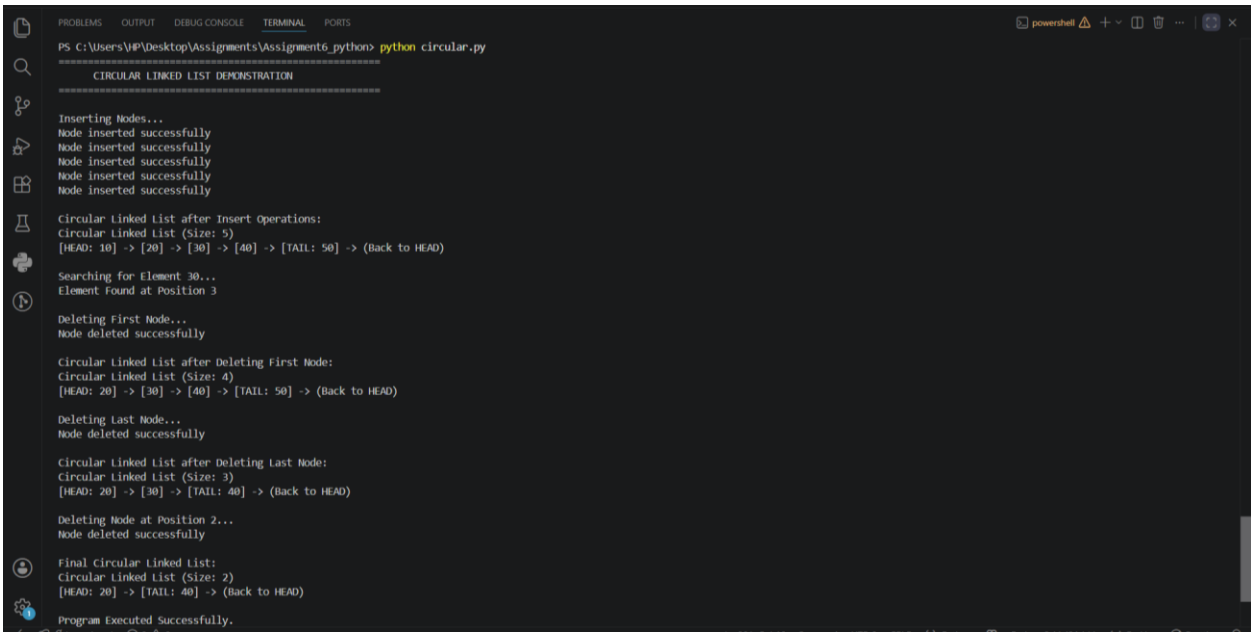


Explanation

This screen demonstrates the implementation of the **Doubly Linked List**. Each node maintains links to both its previous and next nodes, allowing traversal in both forward and backward directions.

The program performs insertion, deletion, and search operations while maintaining the doubly linked structure.

Screenshot 3: Circular Linked List Operations

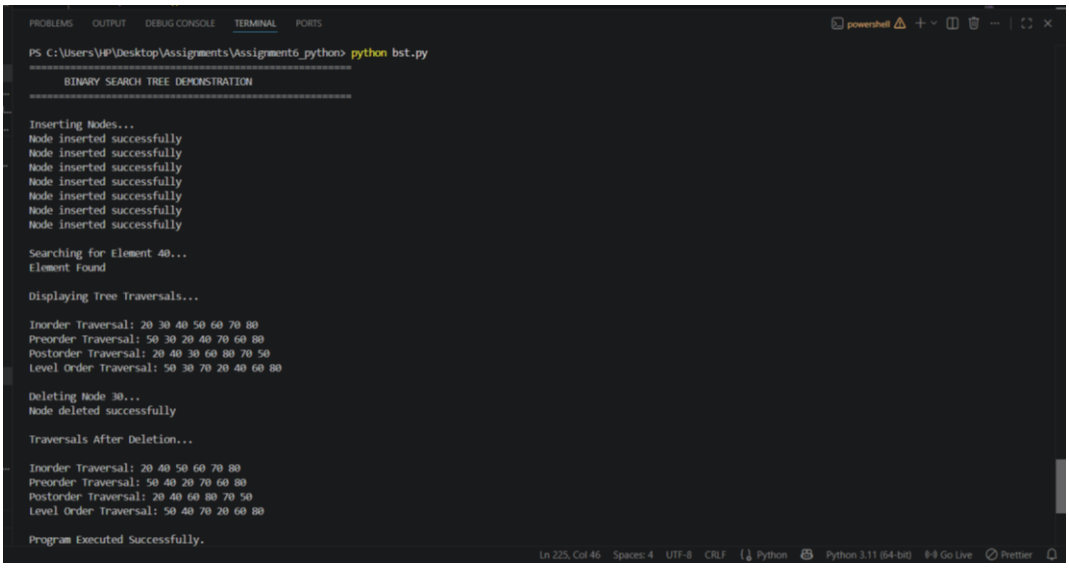


Explanation

This screen demonstrates the implementation of the **Circular Linked List**. The last node is connected back to the first node, forming a circular structure.

The program performs insertion, deletion, and search operations while maintaining the circular connection between nodes.

Screenshot 4: Binary Search Tree Operations



Explanation

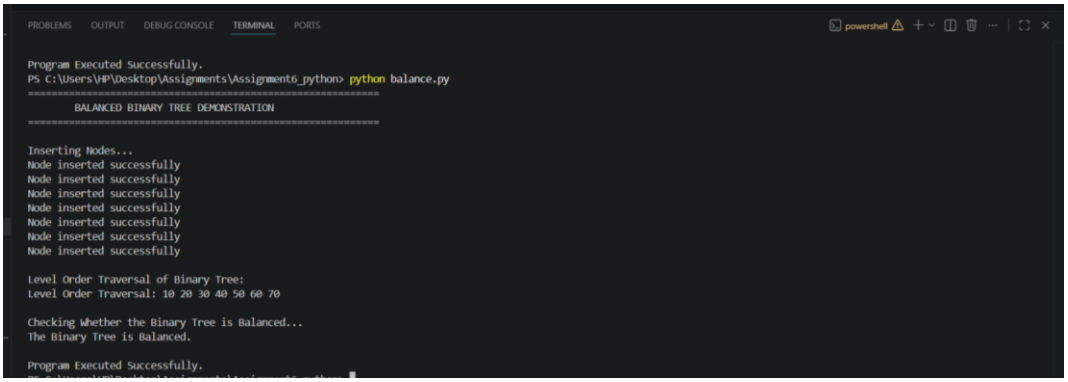
This screen demonstrates the implementation of the **Binary Search Tree (BST)**. The program performs node insertion, searching, and deletion while maintaining the properties of a Binary Search Tree.

It also demonstrates different tree traversal techniques:

- Inorder Traversal
- Preorder Traversal
- Postorder Traversal
- Level-order Traversal

The output verifies the execution of the required BST operations and traversal techniques.

Screenshot 5: Balanced Binary Tree Check



Explanation

This screen demonstrates the algorithm used to determine whether a **Binary Tree is balanced**.

The program calculates the heights of the left and right subtrees of each node and checks their height difference. If the difference satisfies the required balance condition for every node, the tree is identified as balanced; otherwise, it is considered unbalanced.

8. Learning Outcomes

After completing this project, I learned how to:

- Implement different types of Linked Lists, including Singly, Doubly, and Circular Linked Lists.
- Perform insertion, deletion, and search operations on Linked Lists.
- Understand the structure and properties of Binary Search Trees.
- Implement BST operations such as insertion, deletion, and searching.
- Perform Inorder, Preorder, Postorder, and Level-order Traversals.
- Understand how Queue can be used for Level-order Traversal.
- Determine whether a Binary Tree is balanced using recursion.
- Apply recursive logic to tree-related problems.
- Strengthen my understanding of advanced Data Structures and Algorithms.
- Improve logical thinking and problem-solving skills through practical implementation.
- Gain hands-on experience with DSA concepts commonly used in technical interviews

9. Conclusion

The **Linked Lists & Trees Exploration** project was successfully completed using Python. The project demonstrates the implementation of advanced data structures including **Singly Linked List, Doubly Linked List, Circular Linked List, and Binary Search Tree** along with their fundamental operations.

It also includes different tree traversal techniques and an algorithm to check whether a Binary Tree is balanced.

Working on this project provided practical experience with **linear and hierarchical data structures**, strengthened my problem-solving skills, and improved my understanding of important DSA concepts used in software development and technical interviews.